

LoanInNeed Server

Complete Bug & Issue Report

81

Total Tests

45

Pass

25

Fail

11

Warn

Project:	LoanInNeed Server (LoanInNeedServer)
Branch:	testDeploy
Server:	http://localhost:5000
Date:	2026-06-17
Scope:	OTP Auth APIs · KYC APIs · UI Issues · Track Loan

Table of Contents

1. Executive Summary	3
2. OTP Authentication API Bugs	4
2.1. Critical Bugs — OTP APIs	4
2.2. High Severity Bugs — OTP APIs	5
2.3. Medium & Low Severity — OTP APIs	6
3. KYC API Bugs	7
3.2. High Severity Bugs — KYC APIs	7
3.3. Medium Severity Bugs — KYC APIs	9
3.4. Low Severity / Code Quality — KYC APIs	11
4. UI Issues	12
5. Track Loan Issues	13

SECTION 1 — EXECUTIVE SUMMARY

The OTP authentication and KYC APIs are partially functional but contain serious security vulnerabilities that MUST be resolved before going to production.

Key Concerns:

- A hardcoded master bypass code ("261102") exists directly in the source code. Anyone who finds this code can log in as ANY phone number — no real SMS verification is needed. This is a critical authentication bypass.
- A malicious script tag submitted in the loan "purpose" field is stored directly in the database and sent back in API responses — this is a stored XSS attack vector.
- The Aadhaar OTP verification is a complete stub — it accepts ANY input, including an empty body, and returns "Aadhaar verified successfully." There is zero actual identity verification happening.
- No rate limiting is in place anywhere — an attacker can fire thousands of OTP requests or verification attempts with no throttling or lockout.

These issues collectively represent significant security risks, data integrity problems, and poor user experience. None of the critical issues require major architectural changes — they are targeted fixes in specific service files.

SECTION 2 — OTP AUTHENTICATION API BUGS

Endpoints Covered:

- POST /api/auth/phone/request-otp
- POST /api/auth/phone/verify-otp
- POST /api/auth/aadhaar/request-otp
- POST /api/auth/aadhaar/verify-otp

2.1 Critical Bugs — OTP APIs

OTP-C1 · Hardcoded Master Bypass OTP Code

[CRITICAL]

File: services/authService.js ~line 48
Test Cases: T13, T18, T24, T35

The code contains a hardcoded secret OTP value ("261102"). When any user sends this specific code in the verify-otp request, the server skips all OTP verification and immediately creates or logs in the user. There is no check on whether the user actually received an SMS OTP.

- Any developer, tester, or attacker who discovers this code (from code review, version history, or trial-and-error) can log into ANY phone number account without owning the phone.
- Since there is no phone number validation when this bypass is used (see OTP-C2), an attacker can create accounts with garbage phone numbers like "" OR 1=1 --" stored in the database.
- The bypass code can be used unlimited times for any phone number (replay attack confirmed).

Remove the hardcoded bypass entirely. If developers need a testing bypass, create a DEV_OTP_BYPASS_CODE environment variable that is explicitly removed in production deployment. Never hardcode credentials in source code.

OTP-C2 · No Phone Number Sanitisation Before Database Write

[CRITICAL]

File: services/authService.js lines 62–88
Test Cases: T18, T24

When the master bypass OTP is used, the phone number field is never validated. The server accepts and stores ANY string as the user's phone number in the User table.

- Phone: "" OR 1=1 --" !' User created, JWT token returned (200 OK)
- Phone: "+91 " !' User created with whitespace phone, JWT returned (200 OK)

Validate phone against the international E.164 standard format before ANY database operation:

```
/^[+][1-9]\d{7,14}$/
```

2.2 High Severity Bugs — OTP APIs

OTP-H1 · No Rate Limiting on OTP Request Endpoint

[HIGH]

File: services/authService.js lines 12–33
Test Cases: T31

There is no limit on how many OTP requests can be sent for a phone number or from an IP address. All 5 rapid-fire requests in the test returned 200 OK with no throttling.

- An attacker can spam OTP requests to flood the SMS provider (Speqtra API), incurring significant cost.
- Legitimate users can be harassed by unwanted OTP SMS messages.
- Brute-force OTP guessing is possible when combined with unlimited retries.

Install the express-rate-limit npm package and configure:

```
// OTP Request – max 3 per phone per 10 minutes
// OTP Verify – max 5 attempts per phone per 10 minutes, then lock
```

2.3 Medium & Low Severity — OTP APIs

OTP-M1 · Integer OTP Code Type Causes 500 Error

[MEDIUM]

File: controllers/authController.js line 13
Test Cases: T22

When the OTP code is sent as a number (261102) instead of a string ("261102"), the server returns HTTP 500. The code attempts string operations on an integer.

```
const code = String(req.body.code ?? '')
```

OTP-M2 · Aadhaar OTP Is a Complete Bypass — Accepts Anything

[MEDIUM]

File: controllers/authController.js lines 22–35
Test Cases: T27, T28, T29

Both Aadhaar OTP endpoints are stubs returning "success" regardless of any input. An empty body {} on verify-otp returns "Aadhaar verified successfully." This stub must never be exposed in production.

```
if (process.env.AADHAAR_STUB_ENABLED !== 'true') {
  return res.status(503).json({ message: 'Aadhaar verification unavailable' });
}
```

OTP-L2 · Console.log Debug Statements in Production Code

[LOW]

File: controllers/authController.js line 17, middleware/attributionMiddleware.js line 16

Debug log statements like "[DEBUG] Auth Controller - Attribution:" are present in production code. These print sensitive attribution data to server logs. Remove all console.log() calls or replace with logger.debug() which can be silenced by setting the log level in production.

OTP-L4 · JWT Token Contains Personal Data (Phone and Email)

[LOW]

File: utils/jwt.js

The JWT token payload includes the user's phone number and email address. JWT token bodies are base64-encoded (not encrypted) — anyone can decode the token body without the secret key and read the personal data.

```
{ "id": 12705, "email": null, "phone": "+919900000001", "role": "CUSTOMER" }
```

Remove PII from the JWT payload. Store only { id, customUserId, role }. Return only the customUserId (e.g. "LIN12705") — not the raw integer primary key, which reveals database row counts.

SECTION 3 — KYC API BUGS

Endpoints Covered:

- GET /api/kyc
- POST /api/kyc (Full KYC Submission)
- POST /api/kyc/verify-pan
- PUT /api/kyc/employment
- PUT /api/kyc/address

3.2 High Severity Bugs — KYC APIs

KYC-H2 · File Size Error Returns 500 + Multer Stack Trace

[HIGH]

File: routes/kycRoutes.js, middleware/ErrorHandler.js
Test Cases: K38

When a PAN card image larger than 10 MB is uploaded, the server returns HTTP 500 with the full Multer library stack trace including internal file paths and node_modules layout.

Correct behavior: HTTP 400 with message "File too large. Maximum 10 MB allowed."

```
if (err instanceof multer.MulterError && err.code === 'LIMIT_FILE_SIZE') {  
  return res.status(400).json({ message: 'File too large. Maximum 10 MB.' });  
}
```

KYC-H3 · XSS Characters in PAN Number Cause Server Connection Reset / Crash

[HIGH]

File: controllers/kycController.js verifyPAN() function
Test Cases: K37

When a script tag is submitted as the panNumber via multipart form, the server drops the connection entirely with no HTTP response (curl exit code 26 — connection reset). This indicates a server crash or unhandled exception during multipart parsing or the panNumber.toUpperCase() call.

- A single malformed request can crash or hang the server.
- This is a denial-of-service vulnerability.

Validate the panNumber format immediately after reading from req.body:

```
 /^[A-Z]{5}[0-9]{4}[A-Z]{1}$/i  
 // Return 400 immediately if format does not match
```

KYC-H4 · Each POST /api/kyc Creates a New Loan Application (No Idempotency)

[HIGH]

File: services/kycService.js saveFullKYC() function
Test Cases: K18

Every time the same user calls POST /api/kyc (even with identical data), a brand new LoanApplication record is created. Confirmed: User submitted twice !' Application ID 1906 on first call, Application ID 1907 on second call.

- A user with a slow connection who retries will have multiple pending loan applications in the system.
- Loan officers will see duplicate applications without knowing which is correct.
- Downstream LOS integration jobs are created for each duplicate, causing processing errors.

```
existing = findFirst({ where: { userId, status: 'PENDING' } })  
if (existing) !' return existing record instead of creating a new one
```

KYC-H5 · Surepass API Token Hardcoded in Source Code**[HIGH]**

File: services/surepassService.js constructor
Test Cases: K30

The Surepass API authentication JWT token is hardcoded as a fallback value in the source code. If the environment variable SUREPASS_TOKEN is not set, the code uses the hardcoded token.

- The hardcoded token is visible in version control (git history).
- Anyone with repository access can extract the token and make unauthorized PAN verification API calls.
- The token has a 90-day expiry — this is an active, live credential.

```
if (!process.env.SUREPASS_TOKEN) { process.exit(1) }  
// Rotate the currently exposed token immediately.
```

3.3 Medium Severity Bugs — KYC APIs

KYC-M1 · Silent DB Fallback Masks Missing Required Fields**[MEDIUM]**

File: services/kycService.js saveFullKYC(), isPlaceholder() merge logic
Test Cases: K06, K07

If a user already has an employment record in the database and submits POST /api/kyc without required fields like companyName or monthlyIncome, the server silently uses the old database values and returns HTTP 200 OK. A caller who sends an incomplete form gets no error or warning.

Note: This behavior only affects users who already have existing employment records. Fresh users correctly get a 400 error.

Log a warning when a fallback to DB values occurs. Alternatively, require all employment fields on initial full KYC submission and only allow partial updates via the dedicated PUT /api/kyc/employment endpoint.

KYC-M2 · No Maximum Loan Amount Validation**[MEDIUM]**

File: services/kycService.js loanPayload section
Test Cases: K17

Any arbitrarily large loan amount is accepted and stored. Test confirmed that loanAmount = 999999999999 (1 trillion) was accepted with HTTP 200.

- Downstream EMI/interest calculations may overflow or produce incorrect values.
- No business rules are enforced — loan applications for impossible amounts will reach loan officers.

```
if (loanAmount > 500000) return 400 'Loan amount exceeds maximum limit of 15,00,000'
```

KYC-M3 · Invalid Address Type Silently Stored as Null**[MEDIUM]**

File: controllers/kycController.js updateAddress()
Test Cases: K28

When an invalid currentAddressType value (e.g. "INVALID_TYPE") is submitted, the server stores null in the database and returns HTTP 200 OK. The caller has no idea their address type was discarded.

Validate currentAddressType against the known allowed values: RENTED, OWNER_SELF_OR_FAMILY.

KYC-M4 · All 500 Error Responses Expose Stack Traces in Production**[MEDIUM]**

File: middleware/ErrorHandler.js (global error handler)
Test Cases: K15, K19, K23, K38

Every unhandled error returns a response containing error.stack (full JavaScript stack trace with file paths and line numbers) and error.code (internal error codes).

- Stack traces reveal internal file structure and paths.
- Library names and versions are exposed (attackers can look up known vulnerabilities).
- Database ORM query structure is revealed.

```
if (process.env.NODE_ENV !== 'production') { include stack }  
else { return { "status": "error", "message": "Something went wrong" } }
```

3.4 Low Severity / Code Quality — KYC APIs

KYC-L1 · GET /api/kyc Stub Returns HTTP 200 (Should Return 501)

[LOW]

File: controllers/kycController.js getKYC()
Test Cases: K01

The GET /api/kyc endpoint is a stub that returns "KYC details not implemented yet." with HTTP 200, implying the request succeeded. An unimplemented endpoint should return HTTP 501 Not Implemented.

Fix: res.status(501).json({ message: 'Not implemented yet' })

KYC-L3 · Unprofessional Error Message on PAN Validation Failure

[LOW]

File: services/surepassService.js verifyPAN()
Test Cases: K34

When an invalid PAN format is submitted, the error message returned is: "oops Invalid Pan number" — inappropriate for a financial application.

Fix: Change to "Invalid PAN number. Please verify and resubmit."

KYC-L4 · No PAN Format Pre-Validation Before External API Call

[LOW]

File: controllers/kycController.js submitKYC()

Arbitrary strings are sent directly to the Surepass external PAN verification API without first checking if the format is valid. This wastes external API calls and quota on obviously invalid inputs.

```
...  
/^[A-Z]{5}[0-9]{4}[A-Z]$/i - validate before calling external API, return 400 for non-  
matching formats
```

SECTION 4 — UI ISSUES

These are front-end issues identified during manual testing:

UI-1 · Salary Field Allows Scroll / Arrow Key Increment-Decrement

Location: Employment section — Monthly Salary / Income input field

The salary input field accepts scrolling with the mouse wheel and clicking the up/down arrow buttons to increment or decrement the value. Users can accidentally change the salary amount by scrolling over the field.

Fix: Add onWheel={(e) => e.target.blur()} to prevent scroll changes.

- Remove spinner arrows with CSS: input[type=number] { -moz-appearance: textfield }
- input[type=number]::-webkit-inner-spin-button { display: none }

UI-2 · PAN Card Surname Field Cannot Be Filled When PAN Has Only First Name

Location: PAN verification page — Surname / Last Name input field

When a user submits a PAN card that has only a first name and no surname (common on some Indian PAN cards), the surname field remains empty AND the input field appears to be disabled or locked. The user cannot type in the surname, and the form cannot be submitted.

Impact: Users with single-name PAN cards are completely unable to complete KYC verification.

Fix: If the surname field is empty after PAN verification, enable the surname input so the user can manually type their surname. Do not disable the field just because the API returned no surname.

UI-3 · Re-Verification Does Not Update Missing Data from New PAN

Location: PAN verification flow — surname/name field after PAN change

When a user who previously verified a PAN with a surname now changes to a different PAN that has NO surname, the old surname from the previous verification is still shown. The system does not clear the surname.

User's profile shows an incorrect surname that does not belong to their current PAN card.

Fix: When re-verification succeeds with a new PAN, fully replace ALL previously stored PAN details. If the new result has no surname, the stored surname should be cleared/emptied.

SECTION 5 — TRACK LOAN ISSUES

These bugs were identified in the Track Loan feature:

TL-1 · Any Phone Number Returns the Logged-In User's Loan Details

When tracking a loan, entering ANY phone number (including an incorrect or completely unrelated phone number) returns the currently logged-in user's own loan details. No error is shown to indicate the phone is wrong.

Impact: This is a serious data privacy issue — users can attempt to look up other users and keep seeing their own data.

Fix: The lookup must search by the submitted phone number and return 404 Not Found if no matching loan exists. It must NOT fall back to the current user's data.

TL-3 · Address Not Fetched from Aadhaar or PAN Card

The Track Loan / user profile does not auto-populate the user's address from their Aadhaar or PAN card verification data. Users must manually enter address details that should already be available from verified documents.

Fix: After successful PAN/Aadhaar verification, extract the address from the verification response and pre-fill the address fields.

TL-4 · Current Address Type Dropdown Is Missing

The "Current Address Type" dropdown (Rented / Owned) is missing from the Track Loan or address section of the application. Users cannot select their address type.

Fix: Add the Current Address Type dropdown with options: Rented, Owned by Self or Family.

TL-5 · Net Banking Details Link Not Working

The link or button for "Net Banking Details" in the loan application flow is not functional. Clicking it does nothing or leads to an error.

Fix: Implement the Net Banking Details flow or connect the existing link to the correct screen/service.

TL-6 · Loan History "Reapply" Button Not Working

In the Loan History section, the "Reapply" option/button does not work. Users who want to reapply for a loan after a previous application cannot do so through this button.

Fix: Connect the Reapply button to the loan application flow, pre-filling relevant user data where possible.

